Cryptography & Trust Management

Sub-Topics:

Symmetric vs. Asymmetric Cryptography, Hash Functions (MD5, SHA), AES, RSA, SSL/TLS Overview, Public Key Infrastructure (PKI), Digital Signatures, Certificate Authorities (CAs), Cryptographic Key Management & Distribution, Kerberos, X.509 Certificates.

What is Cryptography?

Cryptography is the science of **protecting information** by converting readable data (**Plaintext**) into unreadable form (**Ciphertext**) so that only authorized people can read it.

Why Cryptography is used?

- To **protect passwords**
- To **secure emails & messages**
- To **protect online transactions**
- To **ensure data privacy & integrity**

Basic terms

- **Plaintext** → Original readable data
  Example: HELLO
- **Ciphertext** → Encrypted unreadable data
  Example: X7#@K
- **Encryption** → Converting plaintext to ciphertext
- **Decryption** → Converting ciphertext back to plaintext
- **Key** → Secret value used to encrypt/decrypt data

Symmetric Cryptography

Symmetric Cryptography uses **ONE SAME KEY** for both:

- Encryption
- Decryption

👉 Sender and receiver **both must know the same secret key**.

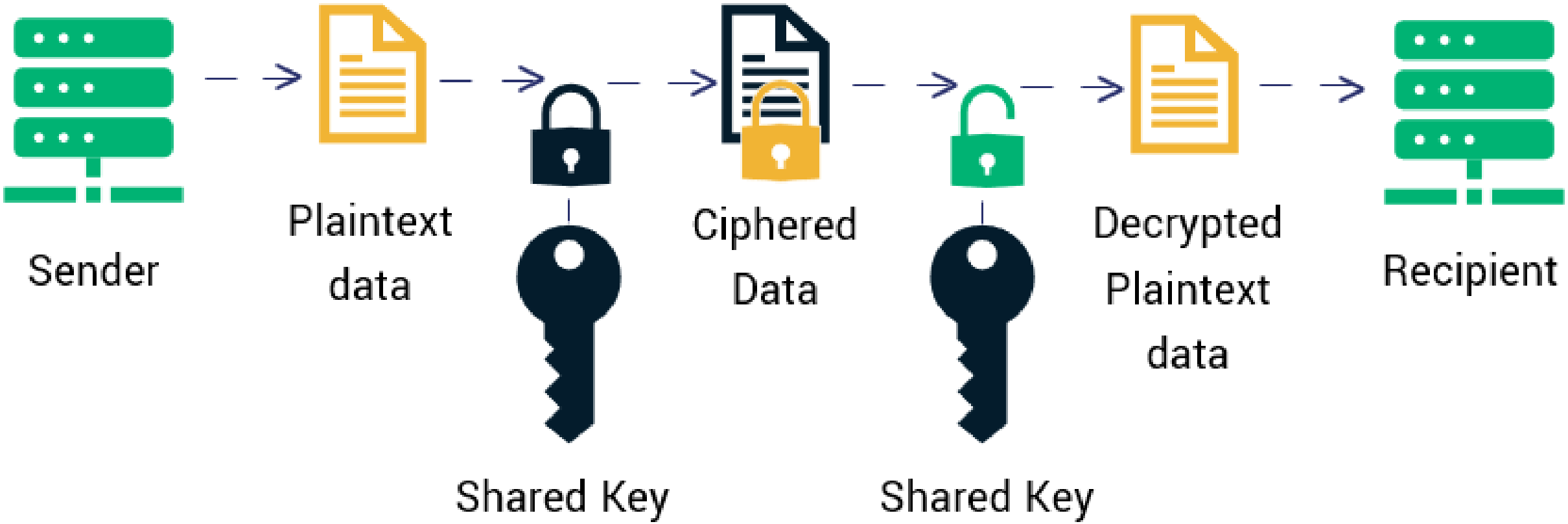
How it works

1.Sender encrypts data using a **secret key**
2.Encrypted data is sent
3.Receiver decrypts data using **the same key**

🧠 Simple Example (Teaching-friendly)

- Key: 3
- Message: HELLO
- Encrypted message: KHOOR
- Receiver uses **same key (123)** to decrypt back to HELLO

Symmetric Encryption



🔧 Real-world Example

- ZIP file password
- Disk encryption
- Wi-Fi password (WPA2/WPA3)

🛠 Popular Symmetric Algorithms

- AES (Advanced Encryption Standard)
- DES (Data Encryption Standard – old)
- 3DES
- Blowfish

✅ Advantages

- Very **fast**
- Efficient for **large data**

❌ Disadvantages

- **Key sharing problem**
- If key is stolen → data compromised

Asymmetric Cryptography

Asymmetric Cryptography uses **TWO DIFFERENT KEYS**:

- **Public Key** → Shared with everyone
- **Private Key** → Kept secret

👉 Data encrypted with **Public Key** can be decrypted only using **Private Key**.

How it works

1.Receiver shares **Public Key**

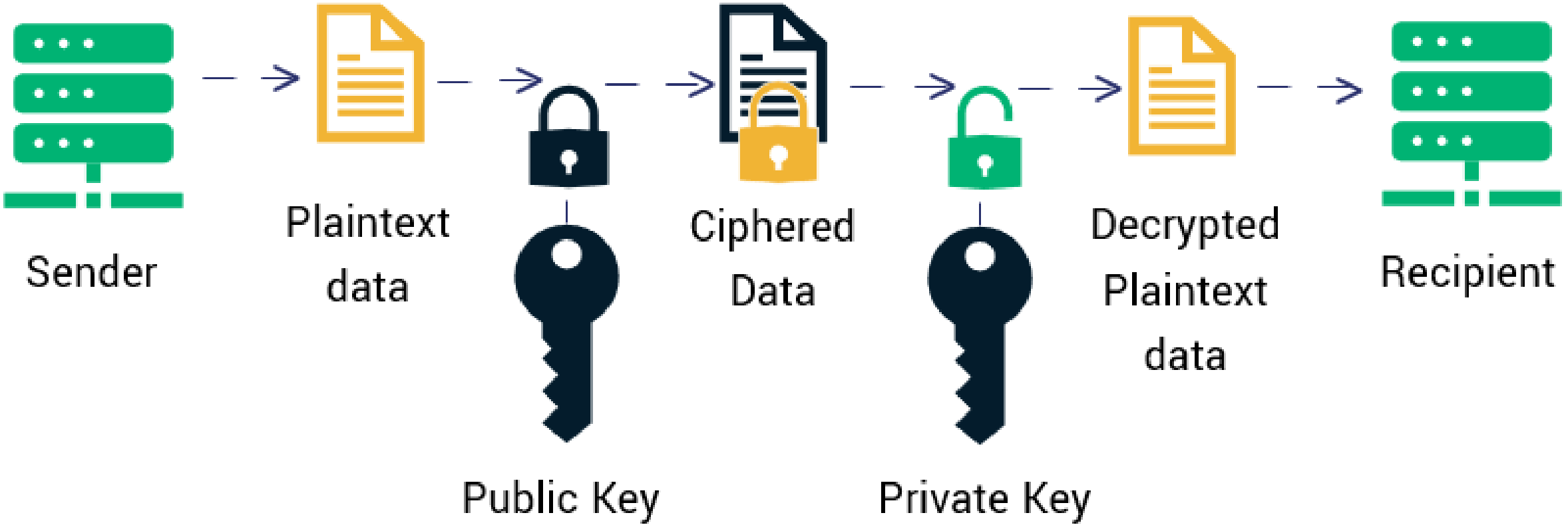
2.Sender encrypts message using Public Key

3.Receiver decrypts using **Private Key**

🧠 Simple Example

- Public Key 🔓 → Lock
- Private Key 🔑 → Actual key
- Anyone can lock the box
- Only owner can open it

Asymmetric Encryption



🔧 Real-world Example

- HTTPS websites
- Email encryption
- Digital signatures
- Online banking

🛠️ Popular Asymmetric Algorithms

- RSA
- ECC (Elliptic Curve Cryptography)
- DSA

✅ Advantages

- **No key sharing problem**
- More **secure for communication**

❌ Disadvantages

- **Slower**
- Not suitable for large data

Symmetric vs Asymmetric (Comparison Table)

Feature	Symmetric	Asymmetric
Keys used	One key	Two keys
Speed	Fast ⚡	Slow 🐢
Security	Medium	High
Key sharing	Required	Not required
Examples	AES, DES	RSA, ECC
Best use	Large data	Key exchange, authentication

1 What is a Hash Function?

A **Hash Function** is a mathematical function that:

- Takes **input data of any size**
- Produces a **fixed-length output** called a **hash value / message digest**

Key Characteristics of Hash Functions

1. **Fixed length output**
2. **One-way function** (cannot be reversed)
3. **Same input → same hash**
4. **Small change in input → completely different hash** (Avalanche effect)
5. **Fast computation**

🔐 MD5 (Message Digest Algorithm 5)

MD5 is a **cryptographic hash function** that converts input data of any length into a **fixed-length 128-bit hash value**, represented as a **32-character hexadecimal string**. It is mainly used to verify **data integrity**.

Family

MD5 belongs to the **MD (Message Digest) family** of hash functions developed by **Ronald Rivest**.

MD Family Members:

- **MD2** – 128-bit hash (obsolete)
- **MD4** – 128-bit hash (broken)
- **MD5** – 128-bit hash (broken)

All algorithms in the MD family are now considered **cryptographically insecure**.

Key Properties

- Produces a **fixed-length output (128-bit)**
- Accepts input of **any size**
- **Deterministic** (same input → same hash)
- **Fast computation**
- **One-way function** (original data cannot be retrieved)
- Vulnerable to **collisions**

Security Status

- ❌ **Not secure for cryptographic use**
- Vulnerable to **collision and preimage attacks**
- Not suitable for **password storage, digital signatures, or secure authentication**
- Deprecated by modern security standards

Real-World Uses

- File integrity verification (non-critical systems)
- Checksums for downloads
- Legacy systems and old applications
- ❌ Not recommended for security-sensitive applications

Input: pass123

```
echo -n "pass123" | md5sum
```

Hash: 482c811da5d5b4bc6d497ffa98491e38

🔐 SHA (Secure Hash Algorithm)

SHA (Secure Hash Algorithm) is a **family of cryptographic hash functions** designed by the **National Security Agency (NSA)** and standardized by **NIST**.
It converts input data of any size into a **fixed-length hash value** and is widely used to ensure **data integrity, authentication, and security**.

SHA Family Members

- **SHA-1** – 160-bit hash (deprecated)
- **SHA-2 Family**
  - SHA-224
  - SHA-256
  - SHA-384
  - SHA-512
- **SHA-3 Family** (Keccak-based)
  - SHA-3-224
  - SHA-3-256
  - SHA-3-384
  - SHA-3-512

Key Properties

- Fixed-length hash output (varies by version)
- Accepts input of any size
- Deterministic (same input → same hash)
- One-way function (irreversible)
- Strong **collision resistance**
- Designed for **high security**
- Slower than MD5 but far more secure

Security Status

- ❌ **SHA-1**: Broken / insecure (collision attacks exist)
- ✅ **SHA-2**: Secure and widely trusted
- ✅ **SHA-3**: Secure, modern, and resistant to future attacks

SHA-2 and SHA-3 are **recommended by NIST** for current cryptographic applications.

Real-World Uses

- Password hashing (with salt / key stretching)
- Digital signatures (SSL/TLS certificates)
- File integrity verification
- Blockchain (Bitcoin uses SHA-256)
- Software distribution verification
- Message authentication (HMAC-SHA256)

Input: pass123

```
echo -n "pass123" | sha256sum
```

Hash: 482c811da5d5b4bc6d497ffa98491e38

🔐 Comparison Within the SHA Family

Feature	SHA-1	SHA-2	SHA-3
Year Introduced	1995	2001	2015
Hash Output Size	160-bit	224, 256, 384, 512-bit	224, 256, 384, 512-bit
Internal Design	Merkle–Damgård	Merkle–Damgård	Sponge Construction
Security Status	❌ Broken	✅ Secure	✅ Highly Secure
Collision Resistance	Weak	Strong	Very Strong
Performance	Fast	Moderate	Slightly Slower
Standard Body	NIST	NIST	NIST
Current Recommendation	❌ Not Recommended	✅ Recommended	✅ Recommended
Use in Modern Systems	No	Yes	Yes (future-proof)

🔍 Detailed Explanation

🔸 SHA-1

- Produces a **160-bit hash**
- Vulnerable to **collision attacks**
- Deprecated by NIST
- ❌ Not safe for security applications

Used earlier in:
Old SSL certificates, legacy systems (now discontinued)

🔸 SHA-2 (Most Widely Used)

Includes:

- SHA-224
- **SHA-256** (most common)
- SHA-384
- SHA-512

Advantages:

- Strong collision resistance
- Secure and widely trusted
- Used in **SSL/TLS, banking, cloud security**

Example use:

- HTTPS
- Digital signatures
- Blockchain (SHA-256)

🔸 SHA-3 (Latest & Future-Ready)

- Based on **Keccak sponge construction**
- Different internal design than SHA-1/SHA-2
- Resistant to future cryptographic attacks

Advantages:

- Very strong security
- Backup standard if SHA-2 is ever weakened
- Used in high-security environments

🧪 Practical Comparison (Linux / Kali)

SHA-1

```
echo -n "data" | sha1sum
```

SHA-256

```
echo -n "data" | sha256sum
```

SHA-3-256

```
echo -n "data" | sha3sum -a 256
```

🔐 AES (Advanced Encryption Standard)

AES (Advanced Encryption Standard) is a **symmetric-key block cipher** used to encrypt and decrypt data securely.
It was selected by **NIST** in 2001 to replace the insecure **DES** family and is now the **global standard for data encryption**.
AES uses the same secret key for both encryption and decryption.

Family

AES belongs to the **modern symmetric encryption (block cipher) family**.

AES Variants (Same Algorithm, Different Key Sizes)

- **AES-128** → 128-bit key
- **AES-192** → 192-bit key
- **AES-256** → 256-bit key

- 🔹 Block size for all AES versions: **128 bits**

Key Properties

- Symmetric-key encryption
- Fixed block size: **128 bits**
- Key sizes: **128 / 192 / 256 bits**
- Fast and efficient (hardware & software)
- Strong resistance to brute-force attacks
- Widely implemented in hardware (CPU, TPM, smart cards)

Security Status

- ✅ **Highly secure and trusted**
- Approved and recommended by **NIST**
- No practical cryptographic attacks when implemented correctly
- Used by governments, banks, military, and enterprises

AES-256 is considered **future-proof** even against quantum-era threats (with proper modes).

Real-World Uses

- Disk encryption (BitLocker, FileVault)
- Wi-Fi security (WPA2 / WPA3)
- Secure messaging apps (WhatsApp, Signal)
- VPNs and SSL/TLS
- Cloud data encryption
- Database and file encryption

Why AES Replaced DES

- DES key size too small → brute-force attacks
- 3DES is slow and inefficient
- AES provides **higher security + better performance**

Comparison with Old Encryption Family (DES & 3DES)

Feature	DES	3DES	AES
Key Size	56-bit	112/168-bit	128/192/256-bit
Block Size	64-bit	64-bit	128-bit
Security	❌ Broken	⚠ Weak / Slow	✅ Strong
Speed	Fast (old)	Very Slow	Very Fast
Brute Force Resistance	❌ No	⚠ Limited	✅ Yes
Current Status	Obsolete	Deprecated	Industry Standard

🔐 RSA Algorithm (Rivest–Shamir–Adleman)

RSA is an **asymmetric (public-key) encryption algorithm** used to securely exchange data and to create **digital signatures**.

It works with **two mathematically related keys**:

- a **Public Key** for encryption
- a **Private Key** for decryption

The security of RSA is based on the **computational difficulty of factoring very large prime numbers**.

Key Properties

- Uses **two different keys** (public & private)
- Public key can be shared openly
- Private key is kept secret
- Supports **encryption, decryption, and digital signatures**
- Slower than symmetric algorithms (like AES)
- Security depends on **large key sizes** (2048-bit or higher)

Security Status

- ✅ Secure when implemented correctly
- ❌ 1024-bit RSA is no longer considered secure
- ✅ 2048-bit and 3072-bit RSA are recommended by standards
- Widely used and approved by **NIST** and international security standards

Real-World Uses

- HTTPS / SSL-TLS communication
- Digital certificates and PKI
- Secure email (PGP, S/MIME)
- Key exchange mechanisms
- Digital signatures and authentication systems

🔒 SSL (Secure Sockets Layer)

SSL (Secure Sockets Layer) is a **cryptographic security protocol** developed to provide **secure communication over the internet**.
It encrypts data transmitted between a **client (browser)** and a **server**, preventing attackers from reading or modifying the data.

⚠️ Important:

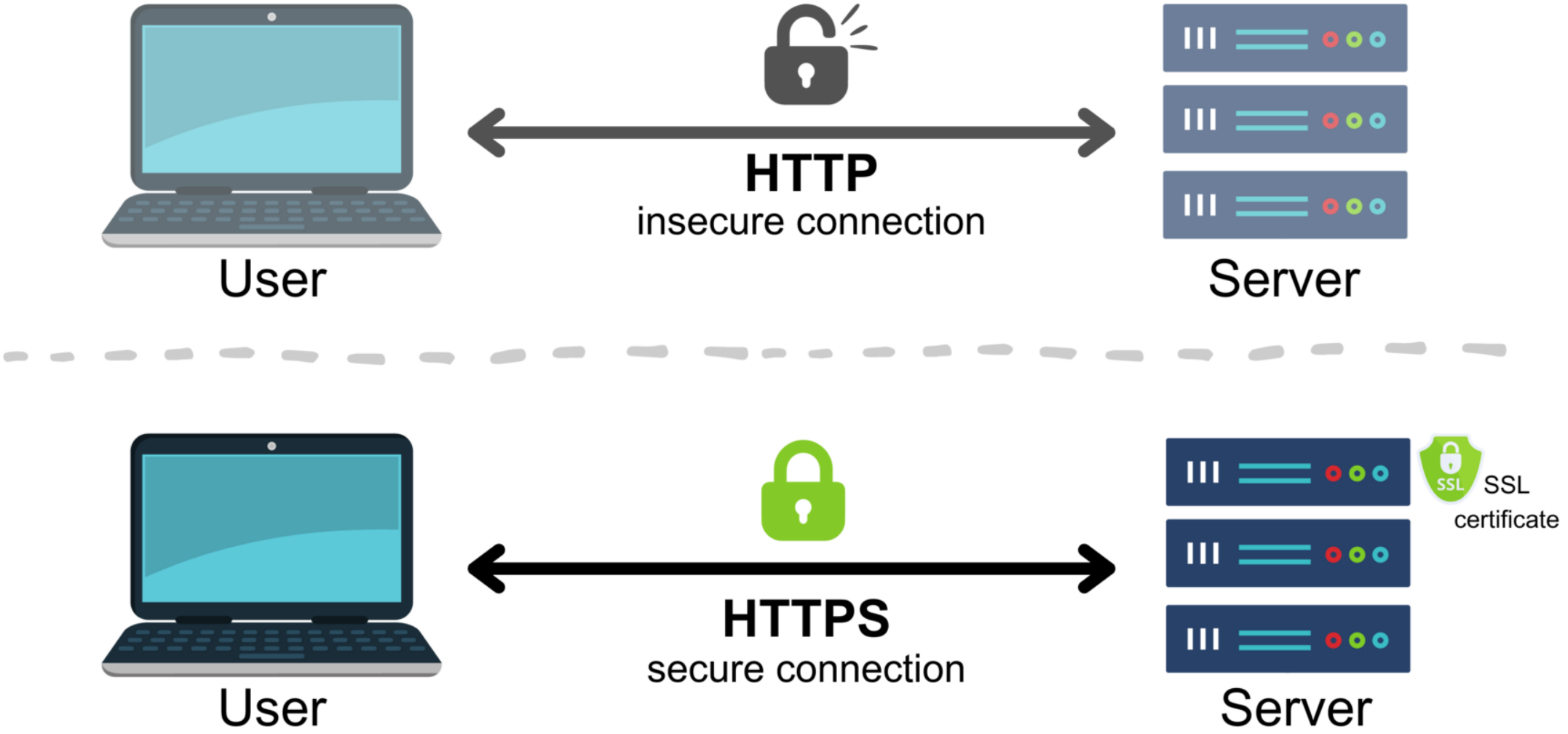
SSL is **deprecated and insecure today**, but it is still mentioned for **academic and historical purposes**.

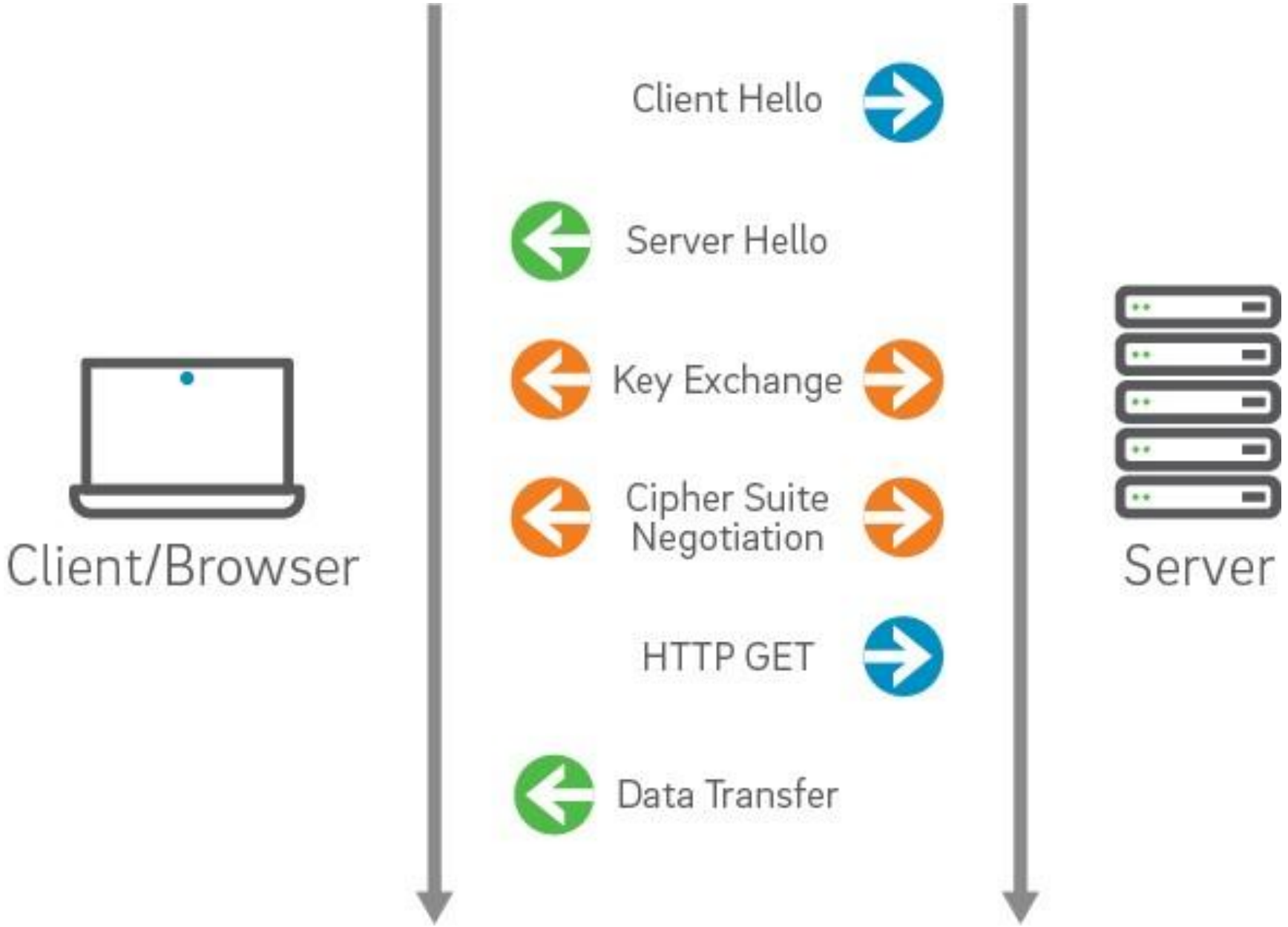
🎯 Objectives of SSL

1. **Confidentiality** – data is encrypted
2. **Authentication** – server identity is verified
3. **Integrity** – data cannot be altered during transmission

What is SSL certificate?

Secure Sockets Layer (SSL)




Client Hello
Server Hello
Key Exchange
Cipher Suite Negotiation
HTTP GET
Data Transfer
Client/Browser
Server

Why SSL Is Not Used Today

- Vulnerable to attacks (POODLE, MITM)
- Uses weak cryptographic algorithms
- Lacks modern security features

🔐 TLS (Transport Layer Security)

TLS (Transport Layer Security) is the **modern, secure successor of SSL**.
It is a **cryptographic protocol** that ensures **privacy, integrity, and authentication** of data exchanged over a network.

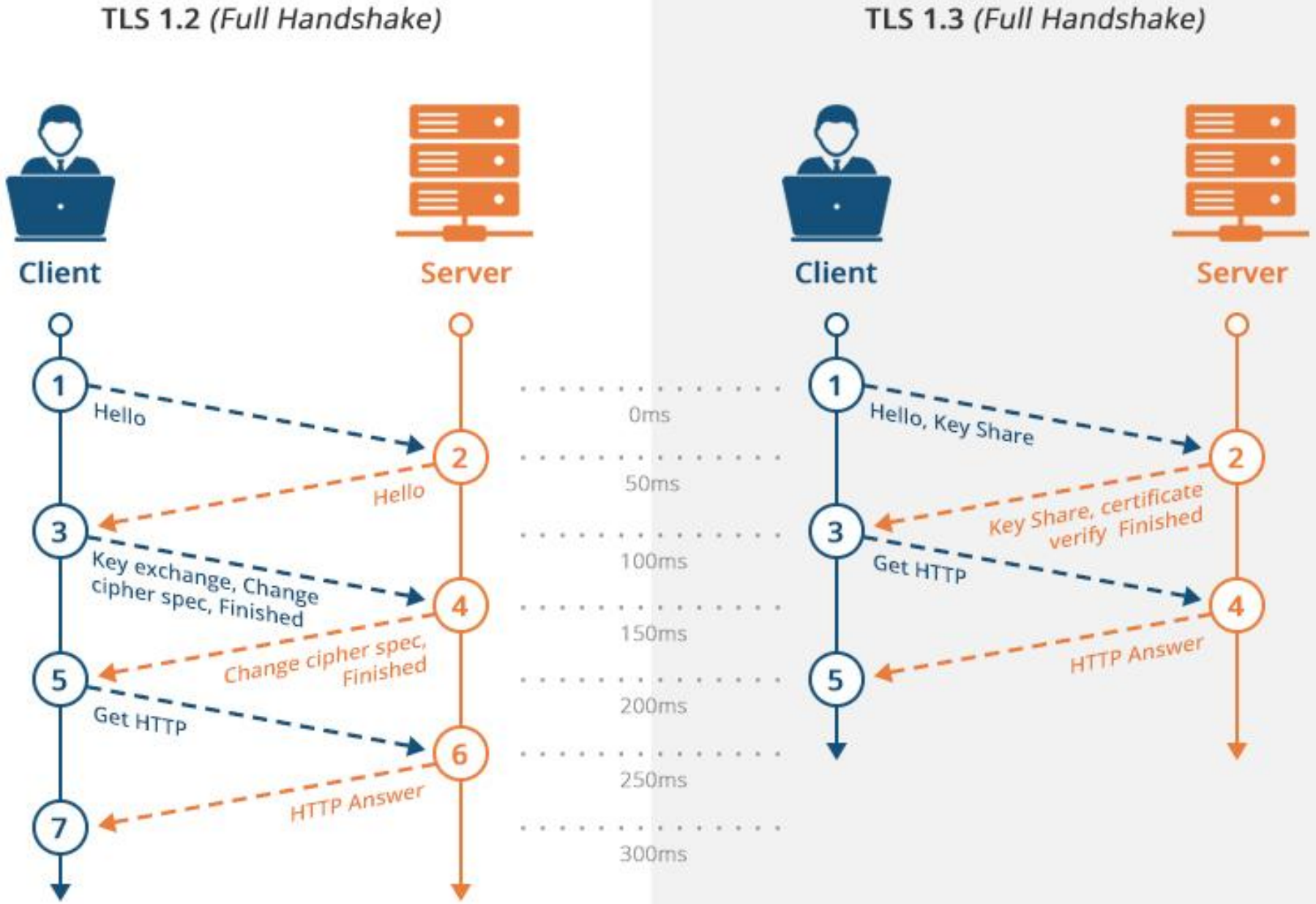
👉 When people say *"SSL certificate"*, they usually mean **TLS certificate**.

🎯 Objectives of TLS

1. **Encryption** – data remains confidential
2. **Authentication** – server (and sometimes client) identity is verified
3. **Integrity** – data is protected from tampering

🌍 Real-World Uses of TLS

- HTTPS websites
- Online banking
- Secure email (SMTP, IMAP)
- APIs & cloud services
- VPNs
- Payment gateways


TLS 1.2 (Full Handshake)
Client
Server
1
Hello
2
Hello
3
Key exchange, Change cipher spec, Finished
4
Change cipher spec, Finished
5
Get HTTP
6
HTTP Answer
7
0ms
50ms
100ms
150ms
200ms
250ms
300ms
TLS 1.3 (Full Handshake)
Client
Server
1
Hello, Key Share
2
Key Share, certificate verify Finished
3
Get HTTP
4
HTTP Answer
5

⚖️ SSL vs TLS (Detailed Comparison)

Feature	SSL	TLS
Full Form	Secure Sockets Layer	Transport Layer Security
Status	Deprecated ❌	Active & Secure ✅
Security	Weak	Strong
Algorithms	MD5, SHA-1	SHA-256, AES
Speed	Slower	Faster
Usage Today	❌ Not used	✅ Used everywhere

📘 What is Public Key Infrastructure (PKI)?

Public Key Infrastructure (PKI) is a **framework of policies, hardware, software, and procedures** used to **create, manage, distribute, store, and revoke digital certificates** and **public–private key pairs**.

👉 PKI enables **secure communication**, **authentication**, **data integrity**, and **non-repudiation** over insecure networks like the Internet.

🎯 Why PKI is Needed

Without PKI:

- Anyone can impersonate a website or user
- No reliable way to verify identity
- Man-in-the-Middle (MITM) attacks are easy

With PKI:

- Identity of users/servers is verified
- Secure key exchange is possible
- Encrypted communication is trusted

🧩 Main Components of PKI

1️⃣ Certificate Authority (CA)

- Trusted organization
- Issues and signs digital certificates
- Root of trust in PKI

📌 Example: Issues SSL/TLS certificates for websites

2️⃣ Registration Authority (RA)

- Verifies identity of users/servers
- Acts as a middle layer between user and CA

3 Digital Certificate

A **digital certificate** binds:

- Identity (domain / user)
- Public key
- CA signature

4 Public & Private Keys

- **Public Key** → Shared openly
- **Private Key** → Kept secret

5 Certificate Repository

- Stores certificates & revocation lists
- Accessible to users and systems

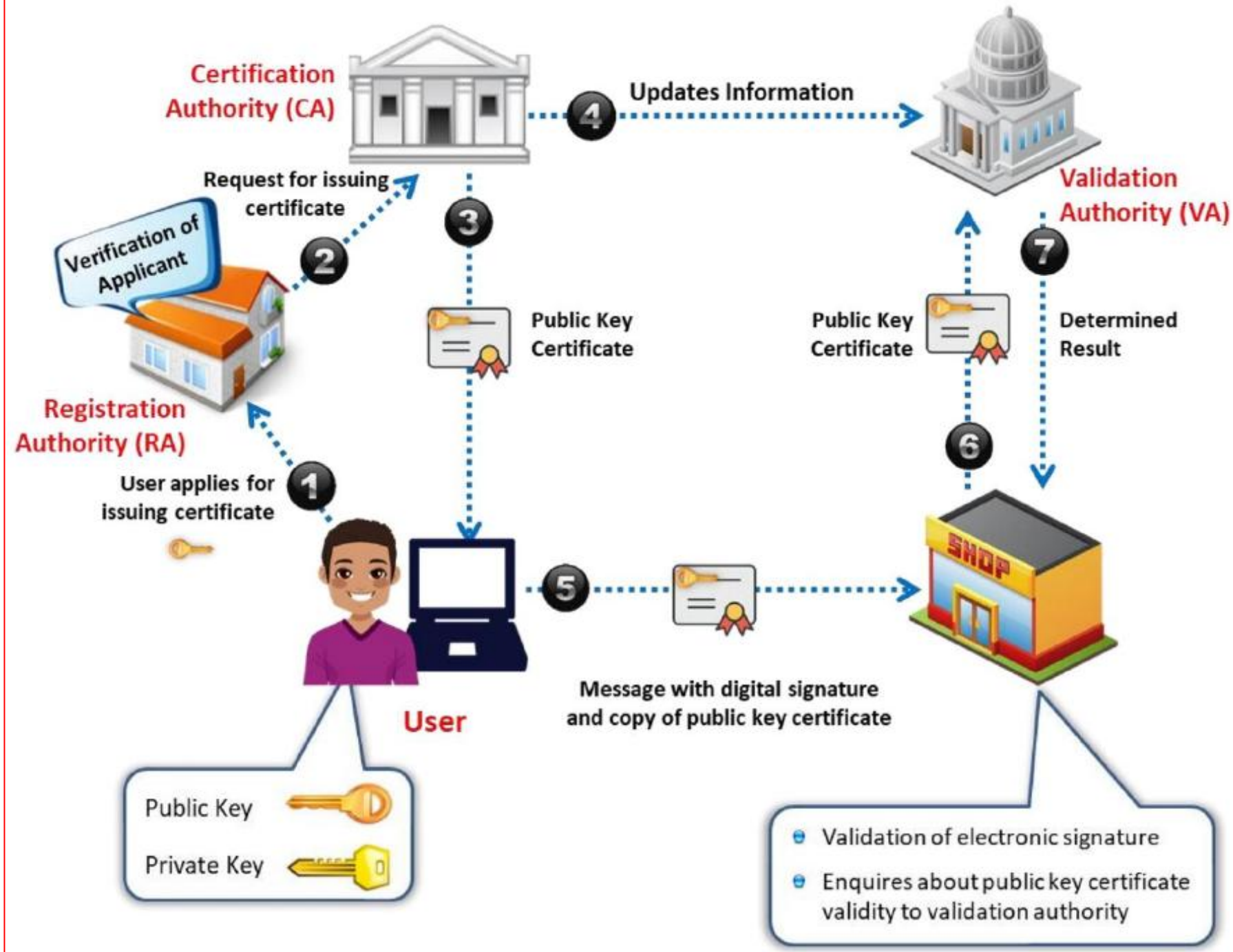
6 Certificate Revocation List (CRL) / OCSP

- Lists revoked certificates
- Used to check certificate validity

⚙️ How PKI Works (Step-by-Step)

🔁 PKI Workflow

1. User generates key pair (public & private)
2. User sends **Certificate Signing Request (CSR)** to CA
3. RA verifies user identity
4. CA issues a **signed digital certificate**
5. Certificate is installed and used
6. Clients verify certificate using CA trust


Certification Authority (CA)
Updates Information
Validation Authority (VA)
Request for issuing certificate
Verification of Applicant
Public Key Certificate
Public Key Certificate
Determined Result
Registration Authority (RA)
User applies for issuing certificate
SHOP
User
Message with digital signature and copy of public key certificate
Public Key
Private Key
Validation of electronic signature
Enquires about public key certificate validity to validation authority

✅ What is a Digital Signature?

A **Digital Signature** is a cryptographic technique used to **prove who sent a message**, **ensure the message wasn't altered**, and **prevent the sender from denying it later**.

In simple words:
Digital Signature = Identity proof + Data integrity + Non-repudiation

It uses **asymmetric cryptography** (private & public keys) and **hash functions**.

🎯 Why Digital Signatures Are Used

Digital signatures guarantee:

1. **Authentication** – confirms who signed the data
2. **Integrity** – ensures data was not modified
3. **Non-repudiation** – signer cannot deny signing later

🧠 How a Digital Signature Works (Step-by-Step)

🔏 Signing Process (Sender)

1.Message → Hash function → **Message hash**
2.Hash is encrypted using **sender's private key**
3.Result = **Digital Signature**
4.Sender sends:
Message + Digital Signature

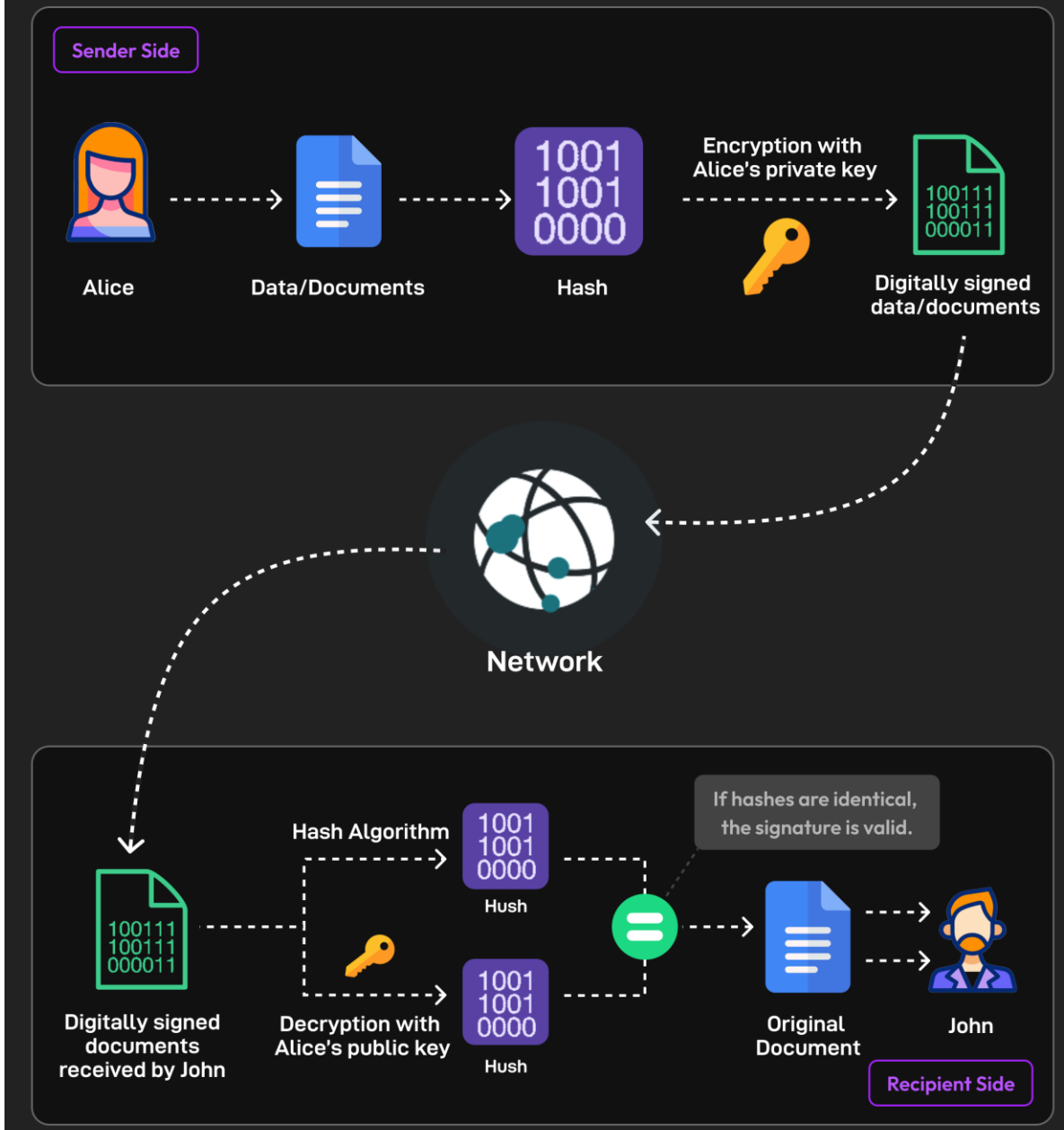
🔓 Verification Process (Receiver)

1.Receiver hashes the received message
2.Receiver decrypts the signature using **sender's public key**
3.If both hashes match → ✅ **Valid Signature**

Common uses:

- Software downloads
- SSL/TLS certificates
- E-documents (PDFs, contracts)
- Emails (S/MIME, PGP)
- Banking & e-governance systems

How Digital Signatures Work?



🧪 Digital Signature Using OpenSSL

- Step 1: Generate Private Key

```
openssl genrsa -out private.key 2048
```

- Step 2: Generate Public Key

```
openssl rsa -in private.key -pubout -out public.key
```

- Step 3: Create a File

```
echo "This is a confidential message" > message.txt
```

- Step 4: Create Digital Signature

```
openssl dgst -sha256 -sign private.key -out signature.bin message.txt
```

✔️ signature.bin is the **digital signature**

- Step 5: Verify Digital Signature

```
openssl dgst -sha256 -verify public.key -signature signature.bin message.txt
```

🔑 Cryptographic Key Management & Distribution

Cryptographic Key Management is the complete process of **generating, storing, distributing, using, rotating, and destroying cryptographic keys** securely.

Key Management = keeping cryptographic keys safe throughout their life

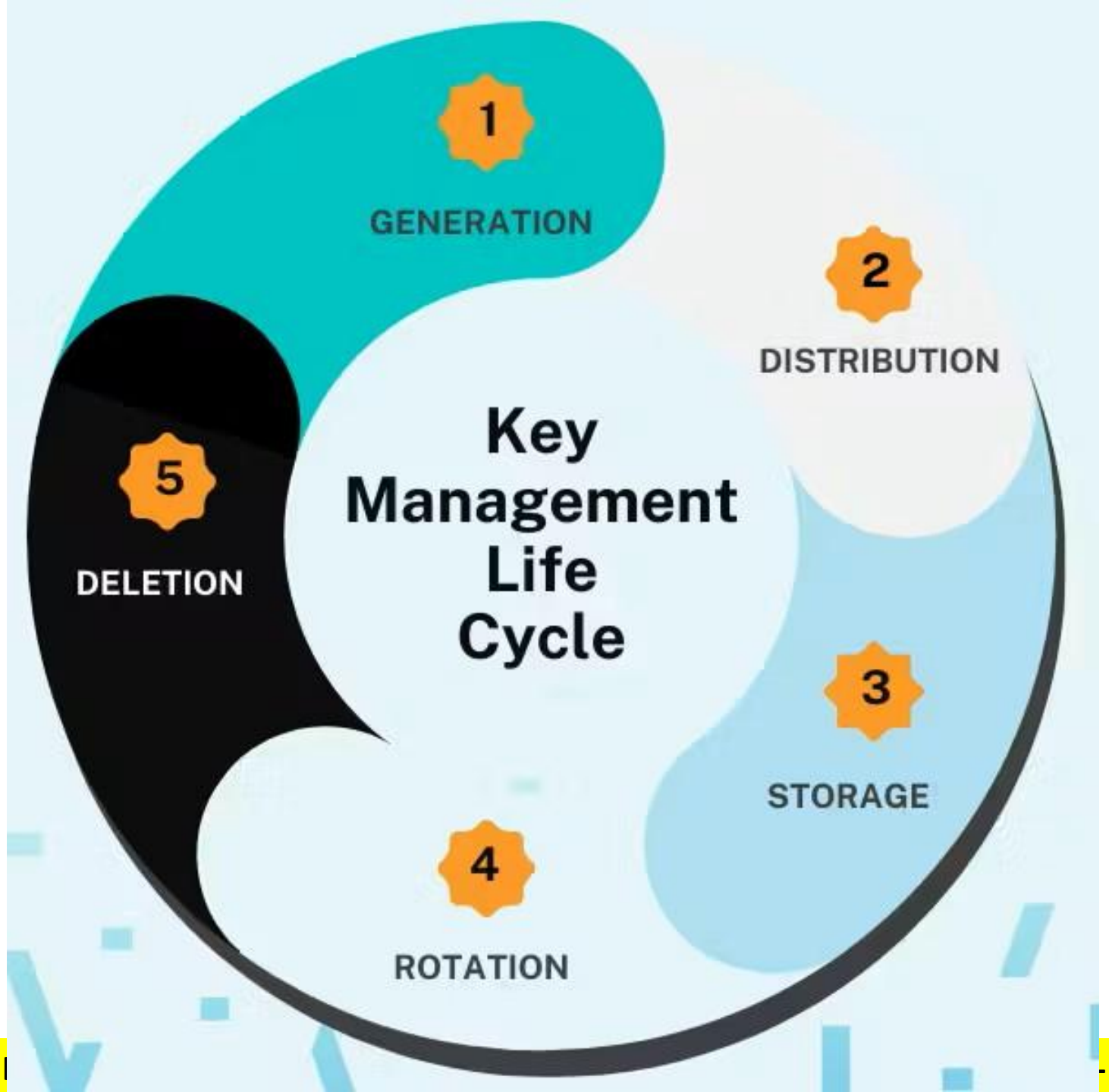
🎯 Why Key Management is Critical

Even the strongest algorithms (AES, RSA, ECC) fail if:

- Keys are stolen
- Keys are reused too long
- Keys are shared insecurely

Key management ensures:

1. **Confidentiality** – only authorized users access data
2. **Integrity** – data not altered
3. **Authentication** – correct identity
4. **Compliance** – banking, govt, ISO standards


1
GENERATION
2
DISTRIBUTION
Key Management Life Cycle
5
DELETION
3
STORAGE
4
ROTATION

1️⃣ Key Generation

- Keys must be generated using **secure random number generators**
- Weak randomness = weak security

Examples

- AES → 128 / 192 / 256-bit keys
- RSA → 2048 / 3072 / 4096-bit keys

2️⃣ Key Storage

Keys must be protected from:

- Theft
- Accidental exposure
- Malware

Secure storage options

- Hardware Security Modules (HSM)
- Trusted Platform Module (TPM)
- Encrypted key vaults
- OS key stores (Windows, Linux)

❌ Never store keys in plain text or source code

3️⃣ Key Distribution (Most Challenging Part)

🔹 1. Symmetric Key Distribution

Same key is used for **encryption & decryption**.

Problem:

How to send the key securely?

Solutions

- Pre-shared keys (PSK)
- Secure physical transfer
- Encrypted key exchange

⚠️ Not scalable for large networks

🔹 2. Asymmetric Key Distribution

Uses **public & private keys**.

✔ Public key can be shared openly

✔ Private key stays secret

Example

- RSA
- ECC

This is the foundation of **PKI & SSL/TLS**

🔹 3. Hybrid Key Distribution (Most Common)

Used in **HTTPS, VPNs, Secure Email**

How it works

1.Public key encrypts a **session key**

2.Session key is symmetric (AES)

3.Data transfer is fast & secure

✔ Best of both worlds

✔ Secure + efficient

🏛️ Key Distribution Using PKI

Public Key Infrastructure (PKI) enables trusted key distribution using **digital certificates**.

Role of PKI:

- Binds public key with identity
- Prevents MITM attacks
- Enables trust via Certificate Authorities

Used in:

- HTTPS
- Digital signatures
- Secure email

🔁 Key Rotation & Renewal

Keys should **not be used forever**.

Why rotate keys?

- Reduce damage if key leaks
- Meet compliance rules
- Improve security hygiene

Example

- SSL certificates: 90 days – 1 year
- AES session keys: per session

❌ Key Revocation & Destruction

Keys must be revoked when:

- Compromised
- Employee leaves
- Certificate expires

Methods:

- Certificate Revocation Lists (CRL)
- OCSP
- Secure key deletion (overwrite memory)

🧪 Practical Example (HTTPS Key Management)

When you open https://bank.com:

1. Server shares **public key (certificate)**
2. Browser verifies trust
3. Browser generates **random AES session key**
4. Session key encrypted with server's public key
5. Secure communication begins 🔒

👉 Public key for distribution
👉 Symmetric key for speed

🔐 Kerberos Authentication Protocol

Kerberos is a secure network authentication protocol that uses **tickets** and a **trusted third party** to authenticate users and services in a network. **without sending passwords**. It is widely used in enterprise environments and is the default authentication protocol for Microsoft's Active Directory (AD).

Kerberos = secure, ticket-based login system

It relies on **symmetric cryptography**, **trusted third party**, and **time-based tickets**.

🧠 Why Kerberos Is Needed

Traditional password authentication problems:

- Password sent over network ❌
- Replay attacks ❌
- Weak trust model ❌

Kerberos solves this by:

- Never sending passwords
- Using **tickets instead of passwords**
- Preventing replay attacks via **timestamps**

🏛️ Core Components of Kerberos

1️⃣ Client (User)

- Person or application requesting access

2️⃣ Server (Service)

- File server, mail server, database, etc.

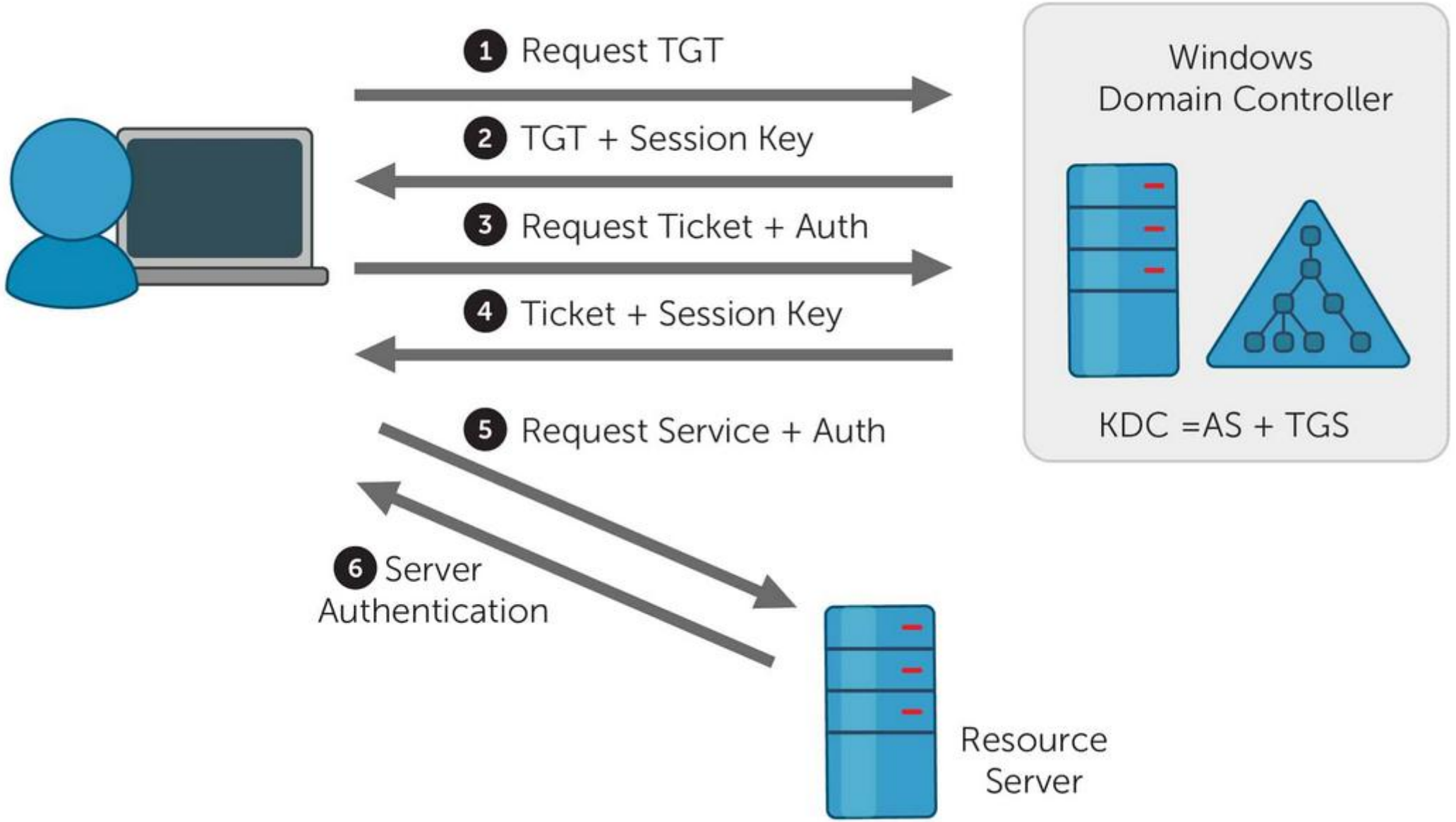
3️⃣ Key Distribution Center (KDC)

Trusted authority that manages authentication.

KDC has **two parts**:

- **AS (Authentication Server)**
- **TGS (Ticket Granting Server)**

KDC concept was developed at **MIT**.


1 Request TGT
2 TGT + Session Key
3 Request Ticket + Auth
4 Ticket + Session Key
5 Request Service + Auth
6 Server Authentication
Windows Domain Controller
KDC =AS + TGS
Resource Server

🔄 How Kerberos Works (Step-by-Step)

🔹 Step 1: Authentication Request (Login)

- User enters username & password
- Client sends request to **AS**
- Password is **never sent**

🔹 Step 2: AS Response

- AS verifies user
- Sends:
  - **TGT** (encrypted with TGS key)
  - Session key (encrypted with user's password hash)

✔️ Only the real user can decrypt it

🔹 Step 3: Request Service Ticket

- Client sends **TGT** to **TGS**
- Requests access to a specific service

🔹 Step 4: TGS Response

- TGS sends **Service Ticket**
- Includes new session key

🔹 Step 5: Access Server

- Client presents Service Ticket to server
- Server verifies ticket
- Access granted ✅

⏱️ Role of Time in Kerberos

Kerberos uses **timestamps** to prevent replay attacks.

Requirements:

- Client & server clocks must be synchronized
- Usually via **NTP**
- Default tolerance ≈ 5 minutes

🖥️ Where Kerberos Is Used

- Windows Active Directory
- Linux enterprise networks
- University systems
- Cloud & enterprise SSO

Windows uses Kerberos by default for **domain authentication**.

🧪 Practical Example (Real-World)

Scenario: Office Network Login

1.Employee logs into system
2.Gets **TGT**
3.Opens file server → uses same TGT
4.Opens email → uses same TGT

✔️ Single Sign-On (SSO)

✅ What is an X.509 Certificate?

An **X.509 certificate** is a **standard digital certificate format** used to **bind a public key to an identity** (person, organization, server, or device).

X.509 certificate = digital ID card for systems & people

It is the backbone of:

- HTTPS (SSL/TLS)
- Digital signatures
- Email security
- VPNs
- PKI systems

🧠 Why X.509 Certificates Are Important

They provide:

1. **Authentication** – who you are
2. **Integrity** – data not altered
3. **Confidentiality** – secure communication
4. **Trust** – via Certificate Authorities (CAs)

🧱 X.509 Certificate Structure (Important Fields)

1️⃣ Version
- X.509 v3 is the most common

2️⃣ Serial Number
- Unique ID assigned by CA

3️⃣ Signature Algorithm
- Algorithm used by CA to sign the certificate (e.g. SHA256 with RSA)

4️⃣ Issuer
- CA that issued the certificate

5️⃣ Validity
- **Not Before** & **Not After** dates

6️⃣ Subject
- Entity that owns the certificate (Domain name / Organization)

7️⃣ Subject Public Key Info
- Public key + key algorithm

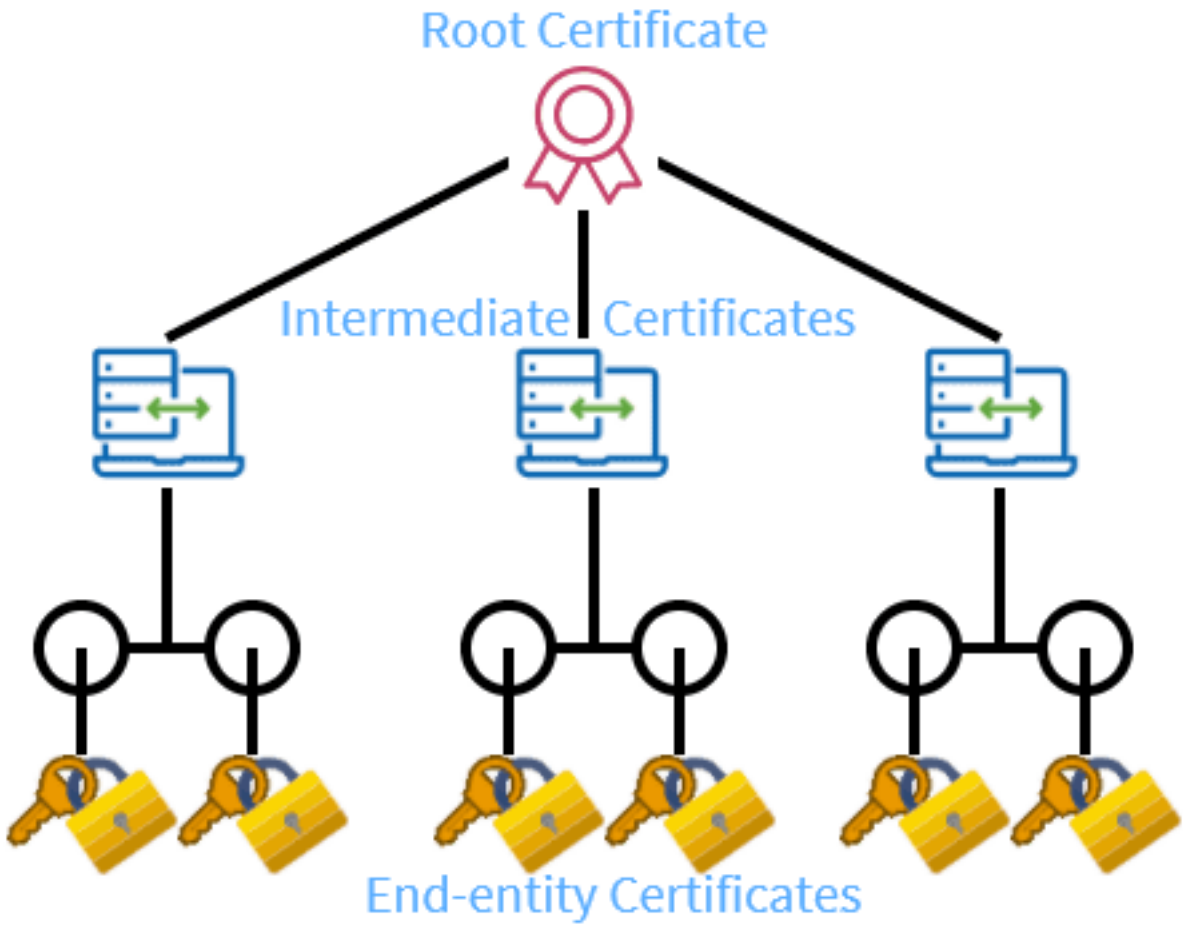
8️⃣ Extensions (X.509 v3)
- Key Usage
- Extended Key Usage
- Subject Alternative Name (SAN)
- Basic Constraints

9️⃣ Digital Signature
- CA’s signature over all fields

🏛️ X.509 and PKI Relationship

X.509 certificates are used inside **Public Key Infrastructure (PKI)**.

Trust chain:



Browsers trust **Root CA certificates** pre-installed in OS/browser.

🌐 Where X.509 Certificates Are Used

- HTTPS websites
- VPN authentication
- Secure email (S/MIME)
- Code signing
- IoT device identity